

Tema 3: Preprocesamiento y Enriquecimiento de Datos con IA

PSU — IA Generativa Aplicada al Análisis de Datos · UNIR · Abril 2026

Índice

1. Introducción y motivación
2. Imputación de valores faltantes: métodos clásicos
3. Imputación con modelos generativos: VAE
4. Imputación con modelos generativos: GAIN
5. Evaluación de imputación: RMSE y norma de Frobenius
6. Generación de variables sintéticas
7. Enriquecimiento semántico con LLM
8. Integración de datos externos con agentes IA
9. Evaluación del impacto del enriquecimiento
10. Automatización del preprocesamiento
11. Preguntas de autoevaluación

1. Introducción y motivación

El preprocesamiento de datos adquiere un nuevo protagonismo con la IA generativa. Ya no se trata solo de limpiar o corregir datos — ahora podemos **completar, ampliar y transformar** la información disponible usando modelos que aprenden la distribución subyacente de los datos.

Este tema opera a nivel de **columnas** dentro de un dataset existente, a diferencia del Tema 2 donde se generaban registros completos desde cero. Las tres grandes tareas son:

Las tres tareas del tema

1. Imputación

: completar columnas que existen pero tienen huecos.

2. Generación de variables sintéticas

: crear columnas nuevas que no existían, inferidas de las existentes.

3. Enriquecimiento semántico

: transformar texto no estructurado en variables numéricas o categóricas.

2. Imputación de valores faltantes: métodos clásicos

La imputación estima entradas ausentes en un dataset. Los métodos convencionales son computacionalmente eficientes pero estructuralmente débiles ante interdependencias complejas.

2.1 Limitaciones fundamentales de la imputación por media

Cuando se imputa con la media global $\bar{x}_j$, se reemplaza cada valor faltante por el mismo escalar, independientemente del contexto de las demás variables del registro. Esto produce tres consecuencias:

Tres fallas estructurales de la imputación por media

1. Reducción de varianza

: la distribución de la variable se aplana artificialmente alrededor de $\bar{x}_j$. La

varianza empírica post-imputación es menor que la verdadera.

2. Sesgo hacia la tendencia central

: aparece un pico artificial en el centro de la distribución — todos los registros imputados colapsan al mismo punto.

3. Ruptura de correlaciones multivariadas

: si las variables están correlacionadas (e.g., edad → ingresos → compras), imputar ingresos con la media global ignora completamente la información disponible en edad y compras del mismo registro.

2.2 Comparativa de métodos clásicos

Método	Descripción	Limitación técnica
Estadístico univariado	Sustituye por media, mediana o moda	No modela relaciones entre variables; reduce la varianza natural
Regresión	Ajusta un modelo para predecir el valor ausente según las demás variables	Supone relación funcional determinista; propaga errores si el modelo está mal especificado
KNN — K-Nearest Neighbors	Imputa por similitud usando los k registros más cercanos en el espacio de atributos	Costoso en alta dimensión; sensible a la elección de k y la métrica de distancia
MICE — Multiple Imputation by Chained Equations	Modela cada variable con ausencias como función del resto, de forma iterativa	Asume relaciones lineales o paramétricas; sensible a la especificación del modelo

Nota clave sobre KNN vs. Media

KNN puede tener mayor RMSE que la media en variables de alta magnitud (e.g., ingresos en el orden de 50.000), porque un error de vecindad se amplifica. Sin embargo, KNN preserva mejor la estructura de correlación multivariada porque el valor imputado proviene de registros con atributos similares — respeta la vecindad local en el espacio de variables.

2.3 El requisito de un método ideal

Un método de imputación ideal debe modelar la **distribución condicional**:

$$P(x_j^{missing} \mid x_1, x_2, \dots, x_{j-1}, x_{j+1}, \dots, x_p)$$

Es decir, debe inferir el valor más coherente para la variable faltante x_j dado el contexto completo del registro — no el promedio global de la población.

3. Imputación con modelos generativos: VAE

VAE — Variational Autoencoder (Autoencoder Variacional)

Modelo generativo que aprende a comprimir datos en un espacio latente continuo y estructurado, y luego reconstruirlos. A diferencia del autoencoder clásico, el codificador no produce un punto sino una *distribución gaussiana* en el espacio latente.

3.1 Arquitectura del VAE

El VAE tiene tres componentes:

Codificador: recibe el vector de entrada $\mathbf{X}$ y produce dos vectores — la media μ

y el logaritmo de la varianza $\log \sigma^2$ de la distribución latente.

Truco de reparametrización: para poder hacer backpropagation a través de un proceso estocástico, el punto latente se construye como:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

donde $\odot$ es el producto elemento a elemento. Esto hace que el gradiente fluya a través

de $\boldsymbol{\mu}$ y $\boldsymbol{\sigma}$ sin pasar por el nodo estocástico $\boldsymbol{\epsilon}$.

Decodificador: recibe $\mathbf{z}$ y reconstruye $\hat{\mathbf{x}}$.

3.2 Función de pérdida: ELBO

El VAE se entrena maximizando el ELBO — Evidence Lower Bound (Cota Inferior de la Evidencia):

$$\mathcal{L}_{ELBO} = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x}|\mathbf{z})] - D_{KL}(q(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$$

El primer término minimiza el error de reconstrucción. El segundo obliga al espacio latente a

parecerse a una gaussiana estándar $\mathcal{N}(\mathbf{0}, \mathbf{I})$, garantizando que el espacio latente sea

continuo, estructurado e interpolable.

Para distribuciones gaussianas, la divergencia KL tiene forma cerrada:

$$D_{KL} = -\frac{1}{2} \sum_{i=1}^d (1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2)$$

3.3 Proceso de imputación con VAE

Cuatro pasos de imputación con VAE

1. Entrenamiento

: el VAE se entrena únicamente con filas completas del dataset, aprendiendo la

distribución conjunta $p(\mathbf{x})$.

2. Codificación parcial

: el registro incompleto se preprocesa reemplazando el hueco con cero (o la media

de la columna). Este vector procesado se pasa al codificador para obtener μ

y σ .

3. Reconstrucción iterativa

: el decodificador genera el vector completo $\hat{\mathbf{x}}$. El valor reconstruido en la

posición faltante reemplaza el cero inicial. El proceso se repite ~15 iteraciones hasta convergencia.

4. Extracción

: se toma únicamente el valor generado para la columna originalmente vacía.

Ventaja adicional del VAE para imputación

Como el espacio latente es probabilístico, se pueden muestrear múltiples valores de

$\mathbf{z}$ para un mismo registro incompleto, obteniendo múltiples imputaciones

plausibles. Esto es especialmente útil en análisis de sensibilidad y simulación de escenarios bajo incertidumbre.

4. Imputación con modelos generativos: GAIN

GAIN — Generative Adversarial Imputation Network (Red de Imputación Adversarial Generativa)

Arquitectura propuesta por Yoon et al. (2018) que reformula la imputación como una tarea de generación adversarial: aprender la distribución condicional de los valores faltantes dado el resto de variables observadas.

4.1 Diferencia conceptual con GAN clásica

En una GAN clásica, el discriminador recibe un dato completo y decide si es real o generado. En GAIN, el discriminador tiene una tarea más granular: decide **posición por posición** cuáles valores de un registro son reales y cuáles fueron imputados por el generador.

4.2 Componentes de GAIN

Máscara explícita M: vector binario donde $M_j = 1$ indica que la variable j es observada y $M_j = 0$ indica que es faltante. Esta máscara se pasa explícitamente al generador como input.

Generador G: recibe el vector de entrada con huecos rellenos por cero, concatenado con la máscara M. Produce los valores imputados para las posiciones faltantes. La reconstrucción final combina datos reales y generados:

$$\tilde{\mathbf{x}} = M \odot \mathbf{x} + (1 - M) \odot G(\mathbf{x}, M)$$

Discriminador D: recibe $\tilde{\mathbf{x}}$ y produce una probabilidad por posición: la probabilidad de que ese valor sea observado (real) vs. imputado (generado).

4.3 Mecanismo de pista (Hint Mechanism)

El elemento clave que distingue a GAIN. El discriminador recibe información parcial sobre qué valores son reales, controlada por el parámetro hint_rate :

$$H_j = M_j \quad \text{con probabilidad} \quad \text{hint_rate}$$

$$H_j = 0.5 \quad \text{con probabilidad} \quad 1 - \text{hint_rate}$$

¿Por qué la incertidumbre forzada mejora el generador?

Cuando $H_j = 0.5$, el discriminador no sabe si la posición j es real o imputada. Para esas posiciones, debe juzgar por **coherencia estadística global** del registro — no por información directa. Esto obliga al generador a producir imputaciones que sean estadísticamente indistinguibles de datos reales, capturando las correlaciones entre todas las variables.

Si el discriminador siempre supiera exactamente qué es real y qué no

($\text{hint_rate} = 1$), simplemente aprendería a mapear entradas a salidas de forma determinista y la dinámica adversarial colapsaría — el generador nunca aprendería a generar imputaciones coherentes con la distribución original.

4.4 Funciones de pérdida de GAIN

Pérdida del discriminador (maximizar capacidad de detección):

$$\mathcal{L}_D = -\mathbb{E}[M \cdot \log D(\tilde{\mathbf{x}}, M, H) + (1 - M) \cdot \log(1 - D(\tilde{\mathbf{x}}, M, H))]$$

Pérdida del generador (engañar al discriminador + fidelidad en posiciones observadas):

$$\mathcal{L}_G = -\mathbb{E}[(1 - M) \cdot \log D(\tilde{\mathbf{x}}, M, H)] + \alpha \cdot \mathbb{E}[(1 - M)(G(\mathbf{x}, M) - \mathbf{x})^2]$$

El hiperparámetro α controla el balance entre engañar al discriminador y producir

valores precisos. Valores típicos: $\alpha = 100$.

5. Evaluación de imputación: RMSE y norma de Frobenius

Para evaluar la calidad de la imputación se utilizan dos métricas complementarias que capturan aspectos distintos de la fidelidad.

5.1 RMSE en celdas imputadas

Mide la precisión puntual: qué tan lejos estuvo el valor imputado del valor real, evaluado únicamente en las posiciones que estaban efectivamente vacías:

$$RMSE = \sqrt{\frac{1}{|\mathcal{M}|} \sum_{(i,j) \in \mathcal{M}} (\hat{x}_{ij} - x_{ij}^{true})^2}$$

donde $\mathcal{M}$ es el conjunto de índices (i, j) que corresponden a celdas

originalmente faltantes.

5.2 Norma de Frobenius de diferencia entre matrices de correlación

Mide la preservación estructural: cuánto se distorsionó la estructura de relaciones entre variables después de imputar.

La **matriz de correlación** del dataset tiene entradas ρ_{jk} — el coeficiente de correlación

de Pearson entre las variables j y k :

$$C_{11} = 1, \quad C_{12} = \rho_{12}, \quad C_{13} = \rho_{13}$$

$$C_{21} = \rho_{12}, \quad C_{22} = 1, \quad C_{23} = \rho_{23}$$

$$C_{31} = \rho_{13}, \quad C_{32} = \rho_{23}, \quad C_{33} = 1$$

La norma de Frobenius de la diferencia entre la matriz verdadera y la imputada es:

$$\|Corr_{true} - Corr_{imp}\|_F = \sqrt{\sum_j \sum_k (c_{jk}^{true} - c_{jk}^{imp})^2}$$

¿Por qué necesitamos ambas métricas?

El RMSE mide *precisión puntual* — celda por celda. La norma de Frobenius mide *preservación estructural* — global. Un método podría tener RMSE razonable pero destruir completamente las correlaciones entre variables (como la imputación por media, que colapsa todas las posiciones al mismo valor). Otro podría preservar bien las correlaciones pero equivocarse bastante en valores puntuales (como KNN en variables de alta magnitud).

5.3 Resultados comparativos del material

Método	RMSE imputado	$\ Corr_{true} - Corr_{imp}\ _F$
GAIN	237.65	0.3041
VAE	258.97	0.3087

MICE	305.31	0.3166
Media	665.15	0.3179
KNN (k=3)	1044.61	0.3113

Lectura de resultados

GAIN y VAE lideran en ambas métricas porque aprenden la distribución conjunta de las variables. MICE es la mejor línea base clásica. KNN tiene el peor RMSE (sensible a la escala de ingresos) pero mejor norma de Frobenius que Media — preserva correlaciones por vecindad local aunque se equivoque en valores absolutos.

Regla práctica para la evaluación

A mayor complejidad de interdependencias y mayor porcentaje de valores faltantes, más ventaja tienen los métodos generativos sobre los clásicos.

6. Generación de variables sintéticas

Distinción conceptual fundamental

Imputación: la columna existe pero tiene huecos — se completa.

Generación de variables sintéticas: la columna no existe — se crea a partir de la información disponible.

El proceso se formaliza como el modelado de una distribución condicional:

$$P(\text{nueva_variable} \mid \text{variables_existentes})$$

Se aprende cómo se comporta una variable que no está en los datos pero que puede inferirse a partir de las demás.

6.1 PCA — Principal Component Analysis

PCA — Análisis de Componentes Principales

Transforma las variables originales en un nuevo conjunto de variables — los *componentes principales* — que son combinaciones lineales ortogonales de las originales, ordenadas por varianza explicada decreciente.

El primer componente principal es:

$$PC_1 = w_1x_1 + w_2x_2 + \cdots + w_px_p$$

donde los pesos $\mathbf{w}$ son los eigenvectores de la matriz de covarianza de los datos — las direcciones de máxima varianza en el espacio multivariado.

Interpretación geométrica: PCA es una rotación de los ejes coordenados hacia las direcciones donde los datos varían más. Los componentes resultantes son variables sintéticas que resumen la varianza conjunta del dataset.

Limitación crítica: PCA solo captura relaciones lineales entre variables. Si las relaciones son no lineales (curvas, interacciones complejas), PCA las pierde.

6.2 Autoencoder: error de reconstrucción como variable de rareza

Un autoencoder aprende a comprimir datos a través de un cuello de botella y reconstruirlos. A diferencia de PCA, usa capas densas con activaciones no lineales (ReLU), capturando relaciones no lineales.

Genera dos tipos de variables sintéticas:

Coordenadas latentes: la representación comprimida de cada registro en el cuello de botella — análogo no lineal de los componentes de PCA.

Error de reconstrucción por registro:

$$AE_rec_error_i = \frac{1}{d} \sum_{j=1}^d (\hat{x}_{ij} - x_{ij})^2$$

Error de reconstrucción como puntuación de rareza

El autoencoder aprende el patrón general del dataset. Cuando encuentra un registro que encaja bien con ese patrón, lo reconstruye con error bajo. Cuando el error es alto, indica que el registro es difícil de reconstruir desde el patrón aprendido — es decir, es un **outlier** o **anomalía**.

Esta variable es especialmente útil en detección de fraude, anomalías clínicas y ciberseguridad, donde el comportamiento atípico es señal de alerta. Es detección de anomalías **no supervisada** — no requiere etiquetas de fraude/no-fraude.

6.3 VAE: coordenadas latentes probabilísticas

El VAE genera coordenadas latentes $\mathbf{z}_1, \mathbf{z}_2, \dots$ que son versiones probabilísticas y más estructuradas que las del autoencoder clásico. Al estar regularizadas por la divergencia KL hacia $\mathcal{N}(0, \mathbf{I})$, el espacio latente es continuo e interpolable.

En la práctica se usa $\boldsymbol{\mu}$ (la media posterior) como variable sintética por registro:

$$VAE_z1_i = \mu_1^{(i)}, \quad VAE_z2_i = \mu_2^{(i)}$$

6.4 Comparativa de métodos para generación de variables sintéticas

Método	Tipo de relación capturada	Variables generadas	Uso principal
--------	----------------------------	---------------------	---------------

PCA	Lineal	PC_1, PC_2, ...	Reducción de dimensionalidad, resumen de varianza
Autoencoder	No lineal	Coordenadas latentes + error de reconstrucción	Representación comprimida + detección de anomalías
VAE	No lineal, probabilística	Coordenadas latentes μ	Variables latentes estructuradas para modelos predictivos
LLM/ Transformer	Semántica contextual	Sentimiento, intención, categoría, resumen	Extracción de valor de texto no estructurado

7. Enriquecimiento semántico con LLM

Enriquecimiento semántico

Transformación de texto no estructurado en variables estructuradas (numéricas o categóricas) mediante LLM — Large Language Models (Modelos de Lenguaje de Gran Escala). Convierte columnas de texto libre en atributos directamente utilizables para modelado.

7.1 Limitación de TF-IDF vs. capacidad de LLM

TF-IDF — Term Frequency-Inverse Document Frequency (Frecuencia de Término-Frecuencia Inversa de Documento) — representa cada texto como un vector de pesos estadísticos. Opera sobre frecuencias de términos sin noción de orden, contexto ni significado.

Ejemplo ilustrativo: las frases "No quiero que me cobren de nuevo" y "Me cobraron de nuevo" comparten casi las mismas palabras. Para TF-IDF producen representaciones casi idénticas. Para un LLM, la primera expresa rechazo y la segunda describe un hecho — sus representaciones son completamente distintas.

Un LLM aprende **representaciones contextualizadas** — cada token adquiere significado distinto según el contexto que lo rodea, gracias al mecanismo de autoatención del Transformer.

7.2 Prompting estructurado para enriquecimiento

El enriquecimiento semántico en un pipeline requiere outputs estables y directamente integrables. Para esto se usan dos configuraciones críticas:

temperature = 0

Hace que el modelo elija siempre el token más probable, produciendo outputs determinísticos y reproducibles. Indispensable en pipelines de producción donde la misma entrada debe producir siempre la misma salida.

Esquema JSON cerrado

Fuerza al modelo a responder únicamente con un JSON que cumple un esquema predefinido. Convierte al LLM en una función determinística: texto libre → variable estructurada integrable directamente al dataframe.

Ejemplo de output con esquema cerrado:

```
{"sentimiento": -1, "intencion_compra": "media"}
```

7.3 Casos de uso comunes

- **Etiquetado semántico:** asignar categorías a registros en texto libre (tipo de queja, nivel de urgencia)
- **Extracción de entidades:** identificar y estructurar nombres, fechas, síntomas, productos
- **Análisis de sentimiento:** cuantificar polaridad emocional del texto
- **Conversión texto → variable:** traducir condiciones textuales en variables computables

Ejemplo del material: enriquecimiento de notas de clientes

Nota 1: "Pedido con retraso, necesito entrega hoy si es posible. Muy urgente."

→ sentimiento: -1 (negativo), intencion_compra: media

Nota 2: "Muy contento; entrega rápida por web. Volvería a comprar."

→ sentimiento: 1 (positivo), intencion_compra: alta

Estas variables permiten priorizar atención (sentimiento = -1 primero), segmentar clientes y mejorar modelos predictivos de churn o retención.

8. Integración de datos externos con agentes IA

Agente de IA

Sistema donde un LLM actúa como componente central de razonamiento, capaz de conectarse con fuentes externas de información. Sigue el ciclo **ReAct — Reason + Act** (**Razonar + Actuar**): interpreta una necesidad, planifica pasos, llama herramientas, procesa respuestas y genera output útil.

8.1 Ciclo ReAct

Observación → Razonamiento → Acción → Observación → ...

A diferencia de un pipeline fijo, el siguiente paso del agente **depende de lo que encuentre** en el paso anterior. Esto habilita razonamiento condicional y adaptativo.

8.2 Herramientas disponibles para agentes

- **APIs externas:** servicios web con datos en tiempo real (clima, calidad del aire, precios financieros, legislación)
- **Motores de búsqueda:** permiten obtener contexto de documentación técnica o páginas web
- **Funciones Python definidas por el usuario:** cualquier función que el agente pueda invocar

Ejemplo del material: pipeline de salud con API externa

Un analista de aseguradora recibe datos de consultas de pacientes. El pipeline automáticamente:

1. Imputa nulos en edad con la media
2. Extrae clasificación de riesgo y sospecha diagnóstica del texto clínico (LLM)
3. Consulta la API Open-Meteo para añadir el índice AQI de calidad del aire por ciudad
4. Ensambla el dataframe enriquecido listo para modelado

Resultado: se pueden detectar correlaciones entre problemas respiratorios y calidad del aire — información imposible de extraer de los datos originales solos.

9. Evaluación del impacto del enriquecimiento

El enriquecimiento no es un fin en sí mismo — debe evaluarse en términos de su impacto directo en el rendimiento predictivo. Variables sintéticas mal calibradas pueden inducir sobreajuste, colinealidad artificial o sesgos implícitos.

9.1 Concepto de uplift

Uplift

Mejora relativa en el rendimiento de un modelo al incorporar variables enriquecidas, comparado con un modelo base sin enriquecimiento. Se calcula entrenando dos modelos idénticos — con y sin las nuevas variables — sobre el mismo conjunto de prueba estratificado e independiente.

9.2 Métricas por tipo de tarea

Tipo de tarea	Métricas relevantes
Clasificación binaria	Accuracy, Precision, Recall, F1-score, ROC-AUC, PR-AUC, Brier Score
Regresión	MSE, RMSE, MAE, R^2
Segmentación / clustering	Silhouette Score, Calinski-Harabasz, Davies-Bouldin

9.3 Riesgo de sobreajuste con variables adicionales

Cada variable nueva añade un grado de libertad al modelo. En datasets pequeños, esto puede producir mejora aparente en entrenamiento que no generaliza al conjunto de prueba. La mejora observada podría explicarse por:

1. Las variables nuevas aportan **señal predictiva real** — capturan información genuinamente útil
2. Las variables nuevas introdujeron **sobreajuste** — el modelo memorizó patrones espurios

9.4 Validación complementaria

Cuatro técnicas de validación complementaria

1. Pruebas de ablación

: eliminación selectiva de cada variable enriquecida para medir su contribución individual. Si al quitar una variable el rendimiento no cambia, esa variable no aportaba señal.

2. Importancia de variables

: valores SHAP (SHapley Additive exPlanations) e importancia por permutación. Cuantifican la contribución de cada variable al output del modelo.

3. Curvas de aprendizaje

: rendimiento del modelo en función del tamaño del conjunto de entrenamiento. Una variable genuinamente útil aporta beneficio consistente en distintos tamaños de dataset.

4. Robustez y explicabilidad

: validar si la variable generada es estable ante cambios leves en la entrada y si puede interpretarse por un experto del dominio.

Regla de oro

Cada variable incorporada debe aportar **señal predictiva** y no solo aumentar la dimensionalidad. Un enfoque basado en uplift que combine validación empírica, interpretabilidad y robustez es clave para una práctica profesional y responsable de la IA generativa.

10. Automatización del preprocesamiento

10.1 Pipeline de preprocesamiento automatizado

Un pipeline es una secuencia estructurada de pasos que transforma datos crudos en un formato listo para modelado. Los pasos típicos son: imputación, normalización, codificación de categóricas, generación de variables sintéticas, enriquecimiento semántico y reducción de dimensionalidad.

Los LLMs contribuyen activamente no solo generando variables o resúmenes semánticos, sino también **escribiendo el código** que implementa todo el pipeline. Esto convierte al LLM en un *copiloto de datos*.

10.2 Distinción pipeline vs. agente

Pipeline con LLM

Para tareas **repetibles, estructuradas y predefinidas**. El flujo no tiene sorpresas — cada paso está especificado de antemano. Ejemplo: clasificar masivamente el sentimiento de 10.000 reseñas con un esquema JSON fijo.

Agente con LLM

Para flujos **dinámicos, contextualizados y dependientes de condiciones externas**. El siguiente paso depende de lo que encuentre el agente en el paso anterior. Ejemplo: investigar la causa raíz de una caída de ventas consultando bases de datos, APIs y documentación según lo que vaya descubriendo.

Son complementarios, no competidores

Un pipeline de producción maduro combina ambos enfoques según la naturaleza de cada tarea.

10.3 Frameworks de automatización

Framework	Fortaleza	Caso de uso ideal	Complejidad
LangChain	Orquestación multi-herramienta: conecta LLMs con		Media-alta

	APIs, bases de datos, funciones Python	Agentes complejos con múltiples herramientas externas	
PandasAI	Interfaz conversacional directa sobre dataframes de pandas	Exploración rápida, limpieza interactiva, prototipado	Baja
DSPy	Optimización automática de prompts usando ejemplos etiquetados	Pipelines robustos en producción a escala	Alta

10.4 Consideraciones de costo en producción

Cada llamada a un LLM externo (OpenAI, Anthropic, Google) consume tokens y genera costo. Las tres estrategias para controlarlo son:

Estrategias de control de costo

1. Selección de modelo adecuado

: no todas las tareas requieren el modelo más potente. Para clasificación de sentimiento con esquema JSON cerrado, un modelo pequeño (e.g., gpt-4o-mini) cuesta ~20x menos que el modelo grande con resultados comparables.

2. Modelos locales

: frameworks como Ollama o HuggingFace permiten correr modelos open source (LLaMA, Mistral, Phi) en infraestructura propia. Costo marginal prácticamente cero después de la infraestructura inicial.

3. Caché de resultados

: si el mismo texto aparece múltiples veces en el dataset, se cachean las respuestas del LLM. LangChain tiene soporte nativo para esto.

Costo del LLM vs. Valor del uplift predictivo

Si el enriquecimiento semántico mejora el F1 de churn de 0.75 a 0.85 y eso se traduce en retener más clientes, el ROI justifica el costo. Si la mejora es marginal, TF-IDF clásico puede ser suficiente y gratuito.

11. Preguntas de autoevaluación

Bloque 1 — Imputación

1. ¿Por qué la imputación por media rompe las correlaciones multivariadas? Explicá el mecanismo con un ejemplo de tres variables correlacionadas.
2. Describí los cuatro pasos del proceso de imputación con VAE. ¿Por qué se repite el proceso iterativamente?
3. En GAIN, ¿qué ocurre si se establece $\text{hint_rate} = 1$? ¿Por qué eso perjudica al generador?
4. KNN tiene peor RMSE que la media en el ejemplo del material, pero mejor norma de Frobenius. ¿Cómo se explica esa aparente contradicción?
5. Escribí la función de pérdida del generador de GAIN e interpretá término por término el rol de α .

Bloque 2 — Generación de variables sintéticas

1. ¿Cuál es la diferencia conceptual entre imputación y generación de variables sintéticas?
2. ¿Por qué el error de reconstrucción de un autoencoder funciona como puntuación de rareza? Dá un ejemplo de aplicación en un dominio real.
3. Comparás PCA y autoencoder para generar variables latentes. ¿En qué tipo de datos preferiría PCA y en cuáles el autoencoder? ¿Por qué?
4. ¿Por qué las coordenadas latentes del VAE son más "estructuradas" que las de un autoencoder clásico?

Bloque 3 — Enriquecimiento semántico y automatización

1. Explicá por qué TF-IDF no puede distinguir entre "No quiero que me cobren" y "Me cobraron". ¿Qué mecanismo del Transformer resuelve esto?
2. ¿Por qué se usa $\text{temperature} = 0$ en enriquecimiento semántico para pipelines de producción?
3. Describí la diferencia entre un pipeline con LLM y un agente con LLM. Dá un ejemplo concreto de cada uno.
4. Listá tres estrategias para controlar el costo de un pipeline con LLM externo en producción.
5. ¿Qué es el uplift y por qué no es suficiente observar que el modelo enriquecido mejora para concluir que las variables nuevas son valiosas?
6. Compará LangChain, PandasAI y DSPy: ¿cuál usarías para exploración rápida de un dataframe nuevo? ¿Cuál para un pipeline robusto en producción? ¿Por qué?

Bloque 4 — Integración conceptual

1. Diseñá un pipeline completo de preprocesamiento para un dataset de tickets de soporte con columnas: id, fecha, texto_queja, tiempo_resolucion (con 15% de nulos), ciudad. El objetivo es predecir si el ticket se resolverá en menos de 48 horas. Especificá qué técnica usarías en cada paso y por qué.
2. ¿En qué situación específica preferirías GAIN sobre VAE para imputación? ¿Y VAE sobre GAIN?